

# Tugas Mandiri 4: Laporan Praktikum Mandiri 4 Machine Learning

Maryam Hasnaa' Syamila - 0110222067

Teknik Informatika, STT Terpadu Nurul Fikri, Depok

E-mail: [hasyamil4@gmail.com](mailto:hasyamil4@gmail.com)

Link Notebook Colab:

[Maryam Hasnaa' Syamila\\_0110222067\\_PraktikumMandiri4.ipynb](#)

Link GitHub Tugas:

<https://github.com/maryamhasnaasyamila/machine-learning/tree/tugas/praktikum/praktikum04>

Link GitHub Praktikum:

<https://github.com/maryamhasnaasyamila/machine-learning/tree/praktikum/praktikum/praktikum04>

## 1. Sinkronisasi Drive, Load Dataset

```
[2] ✓ 22 d
1 # Sinkronisasi dengan drive
2 from google.colab import drive
3 drive.mount('/content/gdrive')
4
5 # Load dataset
6 import pandas as pd
7
8 df = pd.read_csv('/content/gdrive/MyDrive/SEMESTER 7/Machine Learning/
praktikum04/datasets/calonpembelimobil.csv', sep=',')
9
10 # Cetak header data
11 df.head()
```

Mounted at /content/gdrive

|   | ID | Usia | Status | Kelamin | Memiliki_Mobil | Penghasilan | Beli_Mobil |
|---|----|------|--------|---------|----------------|-------------|------------|
| 0 | 1  | 32   | 1      | 0       | 0              | 240         | 1          |
| 1 | 2  | 49   | 2      | 1       | 1              | 100         | 0          |
| 2 | 3  | 52   | 1      | 0       | 2              | 250         | 1          |
| 3 | 4  | 26   | 2      | 1       | 1              | 130         | 0          |
| 4 | 5  | 45   | 3      | 0       | 2              | 237         | 1          |

- Langkah ini bertujuan agar file dataset yang tersimpan di Google Drive dapat diakses langsung dari Colab tanpa perlu diunggah manual.
- Setelah dijalankan, Colab menampilkan tautan otorisasi. Apabila proses berhasil, akan muncul pesan “**Mounted at /content/gdrive**” sebagai tanda bahwa Drive sudah terhubung dan dapat digunakan.

## 2. Import Library Dasar dan Konfigurasi

```
1 # Import library dasar dan konfigurasi
2 import pandas as pd
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import joblib
6
7 # Scikit-learn
8 from sklearn.model_selection import train_test_split, cross_val_score
9 from sklearn.pipeline import Pipeline
10 from sklearn.compose import ColumnTransformer
11 from sklearn.preprocessing import StandardScaler, OneHotEncoder
12 from sklearn.linear_model import LogisticRegression
13 from sklearn.metrics import (
14     accuracy_score, precision_score, recall_score, f1_score,
15     roc_auc_score,
16     confusion_matrix, classification_report, RocCurveDisplay,
17     ConfusionMatrixDisplay
18 )
19
20 # Matplotlib setting
21 %matplotlib inline
22 plt.rcParams['figure.figsize'] = (7, 5)
```

- **pandas** untuk pengolahan dan manipulasi data dalam bentuk tabel (DataFrame),
- **numpy** untuk perhitungan numerik dan operasi berbasis array,
- **matplotlib** untuk membuat grafik visualisasi data,
- **joblib** digunakan untuk menyimpan dan memuat model machine learning yang telah dilatih.

Tahap ini tidak menampilkan output karena hanya mempersiapkan dependensi yang dibutuhkan pada tahap berikutnya.

## 3. Menampilkan Informasi dan Statistik Dataset

```
1 # Informasi kolom dan ringkasan statistik
2 df.info()
3 df.describe(include='all').T
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 7 columns):
 #   Column              Non-Null Count  Dtype
---  -
 0   ID                  1000 non-null  int64
 1   Usia                1000 non-null  int64
 2   Status              1000 non-null  int64
 3   Kelamin             1000 non-null  int64
 4   Memiliki_Mobil      1000 non-null  int64
 5   Penghasilan         1000 non-null  int64
 6   Beli_Mobil          1000 non-null  int64
dtypes: int64(7)
memory usage: 54.8 KB
```

|                | count  | mean    | std        | min  | 25%    | 50%   | 75%    | max    |
|----------------|--------|---------|------------|------|--------|-------|--------|--------|
| ID             | 1000.0 | 500.500 | 288.819436 | 1.0  | 250.75 | 500.5 | 750.25 | 1000.0 |
| Usia           | 1000.0 | 43.532  | 12.672078  | 24.0 | 33.00  | 43.0  | 53.00  | 164.0  |
| Status         | 1000.0 | 1.469   | 1.073402   | 0.0  | 1.00   | 1.0   | 2.00   | 3.0    |
| Kelamin        | 1000.0 | 0.481   | 0.499889   | 0.0  | 0.00   | 0.0   | 1.00   | 1.0    |
| Memiliki_Mobil | 1000.0 | 0.952   | 0.801460   | 0.0  | 0.00   | 1.0   | 2.00   | 4.0    |
| Penghasilan    | 1000.0 | 270.090 | 95.236810  | 95.0 | 187.00 | 258.5 | 352.25 | 490.0  |
| Beli_Mobil     | 1000.0 | 0.633   | 0.482228   | 0.0  | 0.00   | 1.0   | 1.00   | 1.0    |

- Fungsi `df.info()` menampilkan jumlah kolom, jumlah baris, serta tipe data dari setiap kolom.
- Sementara `df.describe(include='all')` memberikan ringkasan statistik (seperti nilai maksimum, minimum, rata-rata, dan frekuensi) untuk semua kolom, baik numerik maupun kategorikal.
- Hasil dari perintah ini membantu mengidentifikasi apakah ada data yang kosong dan memeriksa sebaran nilai pada masing-masing variabel.

#### 4. Pemeriksaan Nilai Unik dan Data Kosong

```
1 # Cek nilai unik untuk tiap kolom & missing values
2 for col in df.columns:
3     print(f"{col:15} | unique: {df[col].nunique():3} | sample unique vals: {df[col].unique()[:10]}")
4 print("\nMissing values per column:\n", df.isna().sum())
```

| ID                                          | Usia                                                | Status                        | Kelamin                   | Memiliki_Mobil                  | Penghasilan                                                   | Beli_Mobil                |
|---------------------------------------------|-----------------------------------------------------|-------------------------------|---------------------------|---------------------------------|---------------------------------------------------------------|---------------------------|
| unique: 1000                                | unique: 44                                          | unique: 4                     | unique: 2                 | unique: 5                       | unique: 314                                                   | unique: 2                 |
| sample unique vals: [ 1 2 3 4 5 6 7 8 9 10] | sample unique vals: [32 49 52 26 45 39 38 29 30 51] | sample unique vals: [1 2 3 0] | sample unique vals: [0 1] | sample unique vals: [0 1 2 4 3] | sample unique vals: [240 100 250 130 237 280 150 143 200 174] | sample unique vals: [1 0] |

```
Missing values per column:
ID          0
Usia        0
Status      0
Kelamin     0
Memiliki_Mobil 0
Penghasilan 0
Beli_Mobil  0
dtype: int64
```

- Kode di atas menampilkan jumlah nilai unik dari setiap kolom beserta beberapa contoh nilainya.
- Selain itu, hasil dari `df.isna().sum()` memperlihatkan jumlah data kosong di setiap kolom.
- Tahap ini berguna untuk mengetahui apakah data memerlukan proses pembersihan atau penanganan nilai yang hilang.

#### 5. Pemilihan Fitur dan Target

```
1 # Pilih fitur dan target. Drop ID (identifier)
2 if 'ID' in df.columns:
3     df = df.drop(columns=['ID'])
4
5 # Tentukan target dan fitur kandidat
6 target_col = 'Beli_Mobil'
7 candidate_features = ['Usia', 'Status', 'Kelamin', 'Memiliki_Mobil',
8                       'Penghasilan']
9 features = [c for c in candidate_features if c in df.columns]
10 print("Features used:", features)
11 print("Target:", target_col)
```

```
Features used: ['Usia', 'Status', 'Kelamin', 'Memiliki_Mobil', 'Penghasilan']
Target: Beli_Mobil
```

- Kolom `ID` dihapus karena hanya berfungsi sebagai identitas dan tidak memiliki pengaruh terhadap hasil prediksi.
- Setelah itu ditentukan kolom mana yang digunakan sebagai **fitur (variabel input)** dan **target (variabel yang ingin diprediksi)**.

- Langkah ini memastikan hanya data yang relevan yang akan digunakan dalam proses pelatihan model.

## 6. Pemeriksaan Ulang Fitur dan Target

```
1 # Periksa missing pada fitur yang dipilih dan tipe datanya
2 print(df[features + [target_col]].isna().sum())
3 print("\nDtypes:\n", df[features].dtypes)
```

```
➦ Usia          0
  Status        0
  Kelamin       0
  Memiliki_Mobil 0
  Penghasilan   0
  Beli_Mobil    0
  dtype: int64

Dtypes:
  Usia          int64
  Status        int64
  Kelamin       int64
  Memiliki_Mobil int64
  Penghasilan   int64
  dtype: object
```

- Setelah fitur dan target dipilih, dilakukan pemeriksaan untuk memastikan tidak ada data yang hilang dan tipe datanya sudah sesuai.
- Output dari kode ini menampilkan jumlah data kosong pada tiap kolom serta tipe datanya (numerik atau kategorikal).
- Jika seluruh kolom memiliki nilai nol pada jumlah data kosong, maka dataset sudah siap digunakan untuk proses pelatihan.

## 7. Penentuan Kolom Numerik dan Kategorik untuk Pre-Processing

```
1 # Tentukan kolom numerik dan kategorikal untuk pra-pemrosesan
2 # Jika suatu kolom bertipe object maka dianggap kategorikal
3 # Bila semua numeric, maka categorical_cols akan kosong ([])
4
5 from sklearn.preprocessing import OneHotEncoder, StandardScaler
6 from sklearn.compose import ColumnTransformer
7
8 # Pisahkan kolom numerik dan kategorikal
9 categorical_cols = [c for c in features if df[c].dtype == 'object']
10 numeric_cols = [c for c in features if c not in categorical_cols]
11
12 print("Numeric cols:", numeric_cols)
13 print("Categorical cols:", categorical_cols)
14
15 # Preprocessor: scale numeric, one-hot encode categorical
16 # drop='first' untuk menghindari multikolinearitas
17 # sparse_output=False digunakan pada scikit-learn versi terbaru (≥1.2)
18 preprocessor = ColumnTransformer(
19     transformers=[
20         ('num', StandardScaler(), numeric_cols),
21         ('cat', OneHotEncoder(drop='first', sparse_output=False),
22          categorical_cols)
23     ],
24     remainder='passthrough'
```

```
➦ Numeric cols: ['Usia', 'Status', 'Kelamin', 'Memiliki_Mobil', 'Penghasilan']
  Categorical cols: []
```

- Kolom bertipe **object** dianggap sebagai data kategorikal, sementara kolom numerik akan dinormalisasi.

- Pendekatan ini memungkinkan model machine learning mengolah semua tipe data secara seimbang dengan cara mengonversi kategori menjadi numerik (melalui *one-hot encoding*) dan menstandarkan skala fitur numerik.

## 8. Pemisahan Data Train dan Data Test

```

1 # Pisahkan data menjadi train dan test set
2 from sklearn.model_selection import train_test_split
3
4 # Pisahkan fitur (X) dan target (y)
5 X = df[features].copy()
6 y = df[target_col].copy()
7
8 # Gunakan stratify untuk menjaga proporsi kelas target
9 X_train, X_test, y_train, y_test = train_test_split(
10     X, y, test_size=0.2, stratify=y, random_state=42
11 )
12
13 print("Train size:", X_train.shape)
14 print("Test size :", X_test.shape)

```

 Train size: (800, 5)  
 Test size : (200, 5)

- Sebanyak **80% data** digunakan untuk pelatihan (*training set*) dan **20% data** untuk pengujian (*testing set*).
- Pembagian ini bertujuan agar performa model dapat dievaluasi secara objektif menggunakan data yang belum pernah dilihat oleh model.
- Tidak ada output langsung, tetapi empat variabel baru terbentuk: *X\_train*, *X\_test*, *y\_train*, dan *y\_test*.

## 9. Membuat Pipeline Data Pre-Processing, Pelatihan Model

```

1 # Bangun pipeline (preprocessing + Logistic Regression) dan latih
2 from sklearn.linear_model import LogisticRegression
3 from sklearn.pipeline import Pipeline
4
5 clf = Pipeline([
6     ('pre', preprocessor),
7     ('logreg', LogisticRegression(
8         max_iter=1000,
9         solver='lbfgs',
10        class_weight='balanced',
11        random_state=42
12    ))
13 ])
14
15 # Latih model pada data training
16 clf.fit(X_train, y_train)
17 print("✅ Model berhasil dilatih.")

```

 ✅ Model berhasil dilatih.

- Dengan pipeline, seluruh tahapan transformasi data (seperti *OneHotEncoder* untuk data kategorikal dan *StandardScaler* untuk data numerik) dapat dilakukan otomatis sebelum model dijalankan.
- Model yang digunakan adalah **Logistic Regression**, cocok untuk kasus klasifikasi biner seperti prediksi "*Beli Mobil (ya/tidak)*".
- Pipeline memastikan data diproses secara konsisten dari awal hingga akhir.

- Model akan mempelajari pola hubungan antara fitur dan target berdasarkan data latih.
- Proses ini menghasilkan model yang sudah terlatih dan siap diuji pada data baru.

## 10. Prediksi dan Evaluasi Model

```

1  # Prediksi dan evaluasi model
2  from sklearn.metrics import (
3      accuracy_score, precision_score, recall_score, f1_score,
4      roc_auc_score, confusion_matrix, classification_report
5  )
6
7  # Prediksi label & probabilitas
8  y_pred = clf.predict(X_test)
9  y_proba = clf.predict_proba(X_test)[:, 1]
10
11 # Hitung metrik evaluasi
12 acc = accuracy_score(y_test, y_pred)
13 prec = precision_score(y_test, y_pred)
14 rec = recall_score(y_test, y_pred)
15 f1 = f1_score(y_test, y_pred)
16 roc = roc_auc_score(y_test, y_proba)
17
18 print(f"Accuracy : {acc:.4f}")
19 print(f"Precision: {prec:.4f}")
20 print(f"Recall : {rec:.4f}")
21 print(f"F1-score : {f1:.4f}")
22 print(f"ROC-AUC : {roc:.4f}")
23
24 print("\nConfusion Matrix:")
25 cm = confusion_matrix(y_test, y_pred)
26 print(cm)
27
28 print("\nClassification Report:")
29 print(classification_report(y_test, y_pred))

```

```

⇒ Accuracy : 0.9300
Precision: 0.9829
Recall : 0.9055
F1-score : 0.9426
ROC-AUC : 0.9768

Confusion Matrix:
[[ 71  2]
 [ 12 115]]

Classification Report:

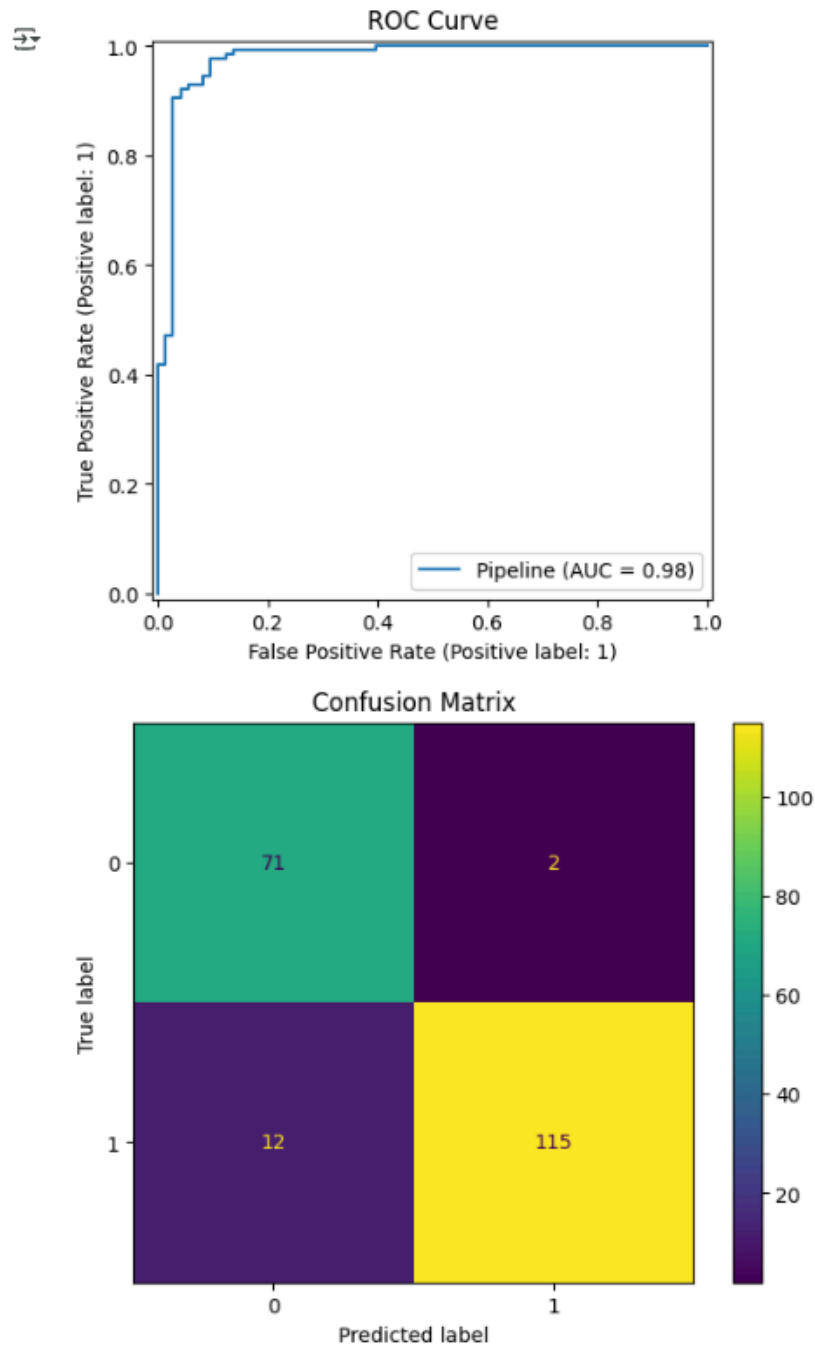
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.86      | 0.97   | 0.91     | 73      |
| 1            | 0.98      | 0.91   | 0.94     | 127     |
| accuracy     |           |        | 0.93     | 200     |
| macro avg    | 0.92      | 0.94   | 0.93     | 200     |
| weighted avg | 0.94      | 0.93   | 0.93     | 200     |

- Nilai **akurasi** (proporsi prediksi benar),
- **Precision**, **Recall**, dan **F1-Score** untuk tiap kelas,
- Matriks konfusi yang menunjukkan distribusi hasil prediksi benar dan salah.
- Hasil ini membantu mengetahui seberapa akurat dan seimbang model dalam melakukan klasifikasi.

## 11. Visualisasi Hasil Prediksi

```
1 # Plot ROC Curve dan Confusion Matrix
2 from sklearn.metrics import RocCurveDisplay, ConfusionMatrixDisplay
3 import matplotlib.pyplot as plt
4
5 RocCurveDisplay.from_estimator(clf, X_test, y_test)
6 plt.title("ROC Curve")
7 plt.show()
8
9 ConfusionMatrixDisplay(confusion_matrix=cm).plot()
10 plt.title("Confusion Matrix")
11 plt.show()
```



- Grafik menampilkan jumlah prediksi benar dan salah dalam bentuk tabel dua dimensi.

- Bagian diagonal menunjukkan prediksi yang benar, sedangkan bagian lainnya menunjukkan kesalahan klasifikasi.
- Visualisasi ini memberikan gambaran yang lebih intuitif tentang performa model.

## 12. Validasi Silang

```
1 # Validasi silang 5-fold untuk estimasi performa
2 from sklearn.model_selection import cross_val_score
3 import numpy as np
4
5 cv_scores = cross_val_score(clf, X, y, cv=5, scoring='accuracy')
6 print("Cross-validation scores:", np.round(cv_scores, 4))
7 print("Mean CV accuracy: {:.4f} ± {:.4f}".format(cv_scores.mean(), cv_scores.std(
  ()))
```

Cross-validation scores: [0.78 0.925 0.955 0.945 0.94 ]  
Mean CV accuracy: 0.9090 ± 0.0652

- Bagian ini digunakan untuk mengevaluasi model dengan cara membagi data menjadi beberapa bagian (fold) dan melatih serta mengujinya secara bergantian.
- Proses ini dilakukan dengan 5-fold cross validation.
- Output berupa lima nilai akurasi dari tiap fold, kemudian dihitung rata-ratanya menggunakan `np.mean(cv_scores)`.
- Validasi silang berguna untuk memastikan model tidak hanya bagus pada data uji tertentu saja, tetapi juga konsisten di berbagai pembagian data.
- Nilai rata-rata akurasi yang tinggi dan stabil menunjukkan model memiliki generalisasi yang baik.

## 13. Melihat Pengaruh Fitur

```
1 # Melihat pengaruh fitur (koefisien Logistic Regression)
2
3 def get_feature_names(column_transformer):
4     feature_names = []
5     for name, trans, cols in column_transformer.transformers_:
6         if name == 'remainder':
7             continue
8         if hasattr(trans, 'get_feature_names_out'):
9             try:
10                 names = list(trans.get_feature_names_out(cols))
11             except Exception:
12                 names = list(cols)
13         else:
14             names = list(cols)
15         feature_names.extend(names)
16     return feature_names
17
18 feat_names = get_feature_names(clf.named_steps['pre'])
19 coeffs = clf.named_steps['logreg'].coef_.flatten()
20
21 feat_df = pd.DataFrame({'Feature': feat_names, 'Coefficient': coeffs})
22 feat_df.sort_values(by='Coefficient', ascending=False).reset_index(drop=True)
```

|   | Feature        | Coefficient |
|---|----------------|-------------|
| 0 | Penghasilan    | 4.568333    |
| 1 | Memiliki_Mobil | 0.078968    |
| 2 | Usia           | -0.045073   |
| 3 | Status         | -0.132093   |
| 4 | Kelamin        | -0.596863   |



- Langkah ini bertujuan untuk mengetahui seberapa besar pengaruh masing-masing fitur terhadap hasil prediksi model.
- Hasilnya berupa tabel yang menampilkan setiap fitur dan nilai koefisiennya.
- Nilai koefisien positif menunjukkan bahwa fitur tersebut berpengaruh **meningkatkan peluang prediksi positif**, sedangkan nilai negatif menunjukkan **pengaruh menurunkan peluang**.
- Dari sini dapat disimpulkan fitur mana yang paling dominan dalam menentukan hasil prediksi.

#### 14. Membuat Dataset Baru

```

1 # Dataset baru (contoh calon pembeli)
2 new_samples = pd.DataFrame([
3     {'Usia': 25, 'Status': 1, 'Kelamin': 1, 'Memiliki_Mobil': 0, 'Penghasilan':
4     300},
5     {'Usia': 40, 'Status': 2, 'Kelamin': 0, 'Memiliki_Mobil': 1, 'Penghasilan':
6     150},
7     {'Usia': 30, 'Status': 1, 'Kelamin': 0, 'Memiliki_Mobil': 0, 'Penghasilan':
8     500},
9     {'Usia': 22, 'Status': 0, 'Kelamin': 1, 'Memiliki_Mobil': 0, 'Penghasilan':
10    80},
11    {'Usia': 50, 'Status': 3, 'Kelamin': 1, 'Memiliki_Mobil': 1, 'Penghasilan':
12    700},
13 ])
14
15 # Urutkan kolom sesuai fitur
16 new_samples = new_samples[features]
17
18 # Prediksi
19 new_pred = clf.predict(new_samples)
20 new_proba = clf.predict_proba(new_samples)[:, 1]
21
22 # Gabungkan hasil
23 result_df = new_samples.copy()
24 result_df['Predicted_Beli_Mobil'] = new_pred
25 result_df['Prob_Beli_Mobil'] = new_proba.round(4)
26
27 result_df

```

|   | Usia | Status | Kelamin | Memiliki_Mobil | Penghasilan | Predicted_Beli_Mobil | Prob_Beli_Mobil |
|---|------|--------|---------|----------------|-------------|----------------------|-----------------|
| 0 | 25   | 1      | 1       | 0              | 300         | 1                    | 0.9390          |
| 1 | 40   | 2      | 0       | 1              | 150         | 0                    | 0.0327          |
| 2 | 30   | 1      | 0       | 0              | 500         | 1                    | 1.0000          |
| 3 | 22   | 0      | 1       | 0              | 80          | 0                    | 0.0004          |
| 4 | 50   | 3      | 1       | 1              | 700         | 1                    | 1.0000          |

- Bagian ini berfungsi untuk menguji model pada data baru yang belum pernah digunakan dalam pelatihan.
- Output dari kode ini adalah hasil prediksi terhadap data baru, misalnya nilai **1** (berarti “**Beli Mobil**” atau “**Jumlah tinggi**”) dan **0** (tidak beli / jumlah rendah) tergantung kasusnya.
- Langkah ini memastikan bahwa model dapat digunakan untuk memprediksi data di dunia nyata.

## 15. Menyimpan Model

```
1 # Simpan model Logistic Regression ke file .joblib
2 import joblib
3
4 joblib.dump(clf, 'logreg_calonpembeli_model.joblib')
5 joblib.dump(clf, '/content/gdrive/MyDrive/SEMESTER 7/Machine Learning/
  praktikum04/models/model_final.pkl')
6 print("✅ Model disimpan sebagai 'logreg_calonpembeli_model.joblib' dan file
  mode_final.pkl")
7
```

✅ Model disimpan sebagai 'logreg\_calonpembeli\_model.joblib' dan file mode\_final.pkl

- Tahap terakhir adalah menyimpan model yang sudah dilatih agar dapat digunakan kembali tanpa perlu melatih dari awal.
- Perintah di atas akan menyimpan model ke dalam file `model_final.pkl` di Google Drive.
- Output berupa pesan konfirmasi penyimpanan file.